

加餐 | 思考题参考答案

潘野 · 云原生基础架构实战课



你好，我是潘野。

今天这次加餐，我为你整理了整门课程的思考题答案，希望对你有所帮助。建议你一定要先自己独立思考、动手编码以后，再对照参考答案，这样学习效果会更好。

因为课程里很多题目都涉及代码，只听音频的话很难理解，所以我建议你直接看文稿。我把每节课的题目和答案放在了一起，每节课的超链接也放在了文稿里，方便你跳转复习。

模块一 基础篇

🔗 第 2 讲

题目

请你尝试参考 Terraform 的官方文档，写一段 Terraform 的代码，在 AWS 上启动一个 EC2 的实例，然后在这个实例中启动 http 服务并对外提供服务。

答案

🔗 <https://github.com/cloudnative-automation/cloud-automation/blob/main/example/webserver/main.cf>

🔗 第 3 讲

题目

这一讲，我提到容器环境中重启主机这有可能会造成一些应用的中断。你知道什么样的情况会造成应用中断么？我们又该如何预防？

答案

如果同学们没有特意为 Deployment 或者 StatefulSet 设置过特殊的调度策略，你可能会遇到多个 pods 被调度到一台机器上的场景。此时重启主机，可能会因为应用容量不足而导致性能下降，甚至服务不可用的情况。

建议你在 Deployment 或者 StatefulSet 中设置反亲和性或者 Pod 拓扑分布约束，以保证 pod 尽可能分散调度，同时启用 PDB，保证最小存活的 pod 数量。

🔗 <https://kubernetes.io/zh-cn/docs/concepts/scheduling-eviction/topology-spread-constraints/>

🔗 第 4 讲

题目

结合上一讲中我们介绍的 Terraform，你能否写一段代码，同时在 AWS 和 Azure 上分别启动一个虚拟机，并在其中启动一个 HTTP 服务？

答案

🔗 <https://github.com/cloudnative-automation/cloud-automation/tree/main/example/webserver>

🔗 第 5 讲

题目

GitOps 并不是一种万能的解决方案，它同样存在一些挑战和限制，你能想到有哪些限制吗？

答案

GitOps 依赖于大量的自动化，这需要投入大量的时间和精力来编写、测试和维护脚本和配置。这可能会对团队的资源和能力提出较高的要求。

GitOps 通常依赖于特定的工具和平台，例如 Kubernetes 和云服务。这可能限制了 GitOps 的适用范围，对于不使用这些工具和平台的环境，可能无法直接应用 GitOps。

模块二 CI/CD 实战篇

🔗 第 6 讲

题目

今天，我们使用了 Terraform 在公有云中启动了一个 Kubernetes 集群。但实际工作中，公司里只有一个集群的情况极少，往往我们面对十多个、甚至上百个 Kubernetes 集群，那么我们要如何管理多个集群的 Terraform 代码呢？

答案

一点小 Tips，这个思考题是今天课程的精髓所在，答案可以在我们课程代码的其他分支里找一找。

🔗 第 7 讲

题目

课程里，我们将 Terraform 状态文件存放在 S3 这步中，S3 存储桶是我们手动在页面上创建出来的，你能否自己写一段 Terraform 代码，通过 GitHub Action 来创建出 S3 存储桶呢？

答案

参考代码仓库中 [🔗 S3 存储桶](#)的代码以及说明。

🔗 第 8 讲

题目

在 terraform init 的这个环节中，我们加载模块代码往往需要花费 5 到 10 分钟。你有哪些手法可以加速加载模块代码么？

答案

一般有两种做法。

第一种是将 terraform 里涉及到的模块预先存在 CI/CD 环境中，比如这一讲放在了 Docker 镜像中，上一讲则是放在 Github Runner 所在的虚拟机中。

第二种做法是将 terraform 里涉及的模块放在公司内网环境中，在 terraform init 的时候直接从内网的服务器中加载。

🔗 第 9 讲

题目

请你根据 [🔗 代码仓库](#)里提供的样例，来完成其他集群组件的部署。

```
1 ---
2 clusterURL: 127.0.0.1
3
4
5 addons:
6
7   namespaces:
8     default: addons-system
9
10  grafana:
11    enabled: false
12    names:
13      app: grafana
14      chart: grafana
15      repoURL: https://grafana.github.io/helm-charts
16      targetVersion: 7.2.4
17
18  ingressNginx:
19    enabled: false
20    names:
21      app: ingress-nginx
22      chart: ingress-nginx/ingress-nginx
23      repoURL: https://kubernetes.github.io/ingress-nginx
24      targetVersion: 4.9.0
```

答案

🔗 <https://github.com/cloudnative-automation/addon-manager/tree/09>

🔗 第 10 讲

题目

请你根据这一讲提供的 Azure Service Operator 讲解，自己动手写一个申请 PostgreSQL 的配置。

答案

📄 复制代码

```
1 apiVersion: database.crossplane.io/v1alpha1
2 kind: PostgreSQLInstanceClass
3 metadata:
```

```
4   name: postgresql-standard
5   namespace: app-project1-dev
6   classRef:
7     kind: RDSInstanceClass
8     apiVersion: database.aws.crossplane.io/v1alpha2
9     name: rdspostgresql-standard
10    namespace: aws-infra-dev
11  ---
12  apiVersion: database.aws.crossplane.io/v1alpha2
13  kind: RDSInstanceClass
14  metadata:
15    name: rdspostgresql-standard
16    namespace: aws-infra-dev
17  specTemplate:
18    class: db.t2.small
19    masterUsername: masteruser
20    securityGroups:
21      - # sg-ab1cdefg
22      - # sg-05adsfkaj1ksdjak
23    size: 20
24    engine: postgresql
25    providerRef:
26      name: example
27      namespace: aws-infra-dev
28    reclaimPolicy: Delete
```

模块三 进阶篇

🔗 第 11 讲

题目

假如你的公司计划将部分业务迁移到公有云，你觉得要如何评估该方案的成本和效益呢？

答案

影响成本的因素可不少，我们需要仔细梳理。

业务类型和需求：不同的业务，对资源的需求也不一样。比如，需要大量计算资源的业务，在公有云上可能比私有云更贵。

资源使用量：用得越多，花得就越多。合理规划资源，才能避免浪费。

数据敏感性：敏感数据需要额外防护，这也会增加成本。

技术支持：想要更完善的技术支持，需要额外付费。

地域：不同地区的云服务价格可能不同。选择合适的地域，可以省点钱。

应用架构：优化应用架构，可以更好地利用公有云的特性，从而降低成本。

公有云服务商的定价模式：不同的服务商，定价模式也不同。货比三家，才能选到最划算的。

成本管理策略：制定合理的成本管理策略，可以有效控制成本。

评估步骤和方法：

1. 明确业务需求：你想把哪些业务搬到公有云？这些业务对资源有什么需求？
2. 分析资源使用量：你目前使用了多少资源？未来可能需要多少资源？
3. 评估数据敏感性：你的数据中有多少敏感数据？需要哪些防护措施？
4. 选择合适的技术支持：你需要什么样的技术支持？愿意为此支付多少费用？
5. 选择合适的地域：你的业务主要面向哪些用户？选择哪个地域可以更好地服务用户？
6. 优化应用架构：如何改造应用架构，才能更好地利用公有云的特性？
7. 公有云服务商的定价模式对比：哪家服务商的定价模式更符合你的需求？
8. 制定成本管理策略：如何监控和控制成本？

评估结论和建议：

根据以上评估，可以得出公有云方案是否比传统方案更具成本效益的结论。如果结论是肯定的，可以制定具体的迁移方案。

以下是一些建议：

在迁移之前，先进行试用，以评估公有云方案的性能和成本。

选择合适的云服务商，并签订有利的合同。

制定合理的资源使用计划，避免浪费。

定期监控和分析成本，并进行必要的调整。

希望这些信息能帮你做出明智的决策！

🔗 第 12 讲

题目

今天的课程里，我提到了 HPA 的一些不足，比如在真正流量突发的时候，有可能会遇到反应慢半拍，无法及时跟上扩展，导致应用服务中断，那你有哪些思路来解决这个问题呢？

答案

这里我提供三个方面的应对思路供你参考。

1. 优化监控指标和策略。

使用更灵敏的监控指标，并结合多维度的指标进行综合判断，以更快速地识别流量突发情况。

调整 HPA 策略，缩短扩缩容的评估周期和步长，使 HPA 能够更迅速地做出反应。

可以考虑使用预测模型，根据历史数据和当前趋势预测未来的流量需求，提前进行扩容。

2. 结合其他扩容手段

使用 Pod 预热机制，提前创建并预热 Pod，以便在需要时快速投入使用。

使用 KEDA 等事件驱动的扩容工具，可以根据事件触发扩容，例如队列长度、消息积压等。

3. 优化应用架构。

使用微服务架构，将应用拆分成多个独立的服务，可以更细粒度地进行扩缩容。

使用无状态服务，可以避免扩容时需要进行数据迁移。

🔗 第 13 讲

题目

你觉得 KEDA 在哪些场景下可以发挥最大价值？

答案

电商平台：在双十一等购物高峰期，电商平台的流量会激增。KEDA 可以根据 HTTP 请求的负载自动扩缩容应用，以确保平台的稳定性和性能。

游戏服务器：游戏服务器的负载会随着玩家数量的增加而增加。KEDA 可以根据玩家登录事件自动扩缩容服务器，以确保玩家的游戏体验。

社交媒体平台：社交媒体平台会产生大量的用户消息。KEDA 可以根据消息队列中的消息数量自动扩缩容应用，以确保消息能被及时处理。

🔗 第 15 讲

题目

回顾一下这个模块的内容，应用层面的水平扩缩容 HPA，垂直扩缩容 VPA，集群层面的扩缩容 Karpenter 与 Cluster Autoscaler，它们适用的场景与缺陷分别是什么。

答案

HPA：应用水平扩容，但是 Kubernetes 原生只支持 CPU/Memory 指标，需要使用 KEDA 或者 Prometheus-Adapter 来配合不同的指标达到水平扩容。

VPA：应用垂直扩容，对应用有要求，能够接受中断。

Cluster Autoscaler：Kubernetes 社区原生的公有云的集群扩缩容方案。

Karpenter: AWS 的集群扩缩容方案。只能用于 AWS, 暂时不能用于其他厂商。

模块四 安全篇

🔗 第 16 讲

题目

在 Kubernetes 集群和容器级别, 具体有哪些安全软件来完成安全防护?

答案

这里我提供一些常见的工具供你参考。

Kubernetes 集群级别的工具包括 Open Policy Agent、NetworkPolicy、Istio 和 Sealed secrets。

容器级别的工具有 Clair 镜像扫描, 以及 Kubernetes Security Context。

🔗 第 17 讲

题目

课程里举了两个场景, 你还知道哪些场景适合使用 vault?

答案

Vault 可以用于各种需要管理敏感数据的场景, 例如以下场景。

证书管理

数据库密码管理

API 密钥管理

SSH 密钥管理

云服务凭证管理

应用程序配置管理

.....

🔗 第 18 讲

题目

不同类型的敏感数据（如数据库凭证、API 密钥等）对租赁时间的需求可能不同。想一想，我们应该如何优化租赁与回收策略呢？

答案

我们一般从事前，事中，事后三个方面入手。

事前

首先，对所有敏感数据进行分类。根据数据的敏感性、使用频率和业务影响进行分级。

然后，根据数据的分类和风险评估，为不同类型的数据定制租赁时间。例如，对于经常变更的临时访问令牌，设置较短的租赁时间；对于稳定使用的 API 密钥，设置较长的租赁时间。

事中

接下来设置警报机制，当租赁即将到期或检测到异常使用模式时，自动通知系统管理员和数据所有者。对于到期或不再使用的租赁，自动化回收过程，确保敏感数据不被遗留在系统中。

事后

审计与合规：确保所有回收操作都有审计记录，以支持合规性要求和安全审计。

定期评审：定期评审和更新租赁与回收策略，以适应新的业务需求和技术变化。

🔗 第 19 讲

题目

除了 Cert-Manager，你还有哪些手段可以从 Vault 中自动获取证书？

答案

除了使用 Cert-Manager，还有多种其他手段可以从 HashiCorp Vault 中自动获取和管理 TLS 证书，适用于不同的应用和环境。以下是一些常见的方法。

Vault Agent

Vault Agent 是一个预配置的客户端守护进程，可以自动化从 Vault 获取证书的过程。它可以配置为自动认证并续订密钥和证书，不仅限于 Kubernetes，也适用于其他各种环境。

Terraform

使用 Terraform 管理 Vault 资源可以自动化证书的配置和部署。Terraform 提供了一种声明式的方式来管理和编排 IT 资源，包括 TLS 证书。通过编写 Terraform 配置文件，我们可以自动从 Vault 请求证书，并将其部署到需要的环境中。

Ansible

Ansible 是一个自动化平台，可以用来自动化证书的部署和管理。通过编写 Ansible playbook，可以从 Vault 中获取证书，并配置到各种服务器和应用中。Ansible 的 Vault 模块可以简化这一过程。

🔗 第 20 讲

题目

请你用 Gatekeeper 定义一个策略，规则是容器启动的时候禁止使用 host network。

答案

 复制代码

```
1 apiVersion: templates.gatekeeper.sh/v1
2 kind: ConstraintTemplate
3 metadata:
4   name: denyhostnetwork
5 spec:
6   crd:
7     spec:
8       names:
9         kind: DenyHostNetwork
10      validation:
11        legacySchema: true
12        openAPIV3Schema:
13          properties:
14            hostNetwork:
15              type: boolean
16            max:
17              type: integer
18            min:
19              type: integer
20      targets:
21      - rego: |
22        package hostnetwork
23
24        violation[{"msg": msg, "details": {}}] {
25          input_share_hostnetwork(input.review.object)
26          msg := sprintf("hostNetwork is not allowed, pod: %v", [input.review.objec
27        ]
28
29        input_share_hostnetwork(o) {
30          o.spec.hostNetwork
31        }
32
33        input_containers[c] {
34          c := input.review.object.spec.containers[_]
35        }
36
37        input_containers[c] {
38          c := input.review.object.spec.initContainers[_]
39        }
40
41        input_containers[c] {
```

```
42         c := input.review.object.spec.ephemeralContainers[_]
43     }
44     target: admission.k8s.gatekeeper.sh
```

 复制代码

```
1  apiVersion: constraints.gatekeeper.sh/v1beta1
2  kind: DenyHostNetwork
3  metadata:
4    name: deny-hostnetwork
5  spec:
6    enforcementAction: deny
7    match:
8      excludedNamespaces:
9        - kube-system
10     kinds:
11       - apiGroups:
12         - ""
13     kinds:
```

好，以上就是这次加餐的全部内容。也欢迎你继续在留言区参与交流和讨论。

AI智能总结

1. KEDA 在电商平台、游戏服务器和社交媒体平台等场景下可以发挥最大价值，通过根据负载自动扩缩容应用来确保平台稳定性和性能。
2. HPA、VPA、Cluster Autoscaler和Karpenter 分别适用于应用水平扩容、应用垂直扩容、集群扩缩容以及AWS 的集群扩缩容，但各自存在一些缺陷和限制。
3. Kubernetes 集群级别的安全软件包括 Open Policy Agent、NetworkPolicy、Istio 和 Sealed secrets，容器级别的工具有 Clair 镜像扫描和 Kubernetes Security Context。
4. Vault 适用于证书管理、数据库密码管理、API 密钥管理、SSH 密钥管理、云服务凭证管理和应用程序配置管理等场景。
5. 优化租赁与回收策略需要从事前、事中和事后三个方面入手，包括对敏感数据进行分类、设置警报机制和审计与合规等措施。
6. 除了 Cert-Manager，还可以使用 Vault Agent、Terraform 和 Ansible 等手段从 HashiCorp Vault 中自动获取证书。
7. Gatekeeper 可以通过定义策略来禁止容器在启动时使用 host network，通过 ConstraintTemplate 和 DenyHostNetwork 实现该策略。

精选留言

由作者筛选后的优质留言将会公开显示，欢迎踊跃留言。